



**TECNOLÓGICO
NACIONAL DE MÉXICO**



Instituto Tecnológico de Tepic

Ingeniería en Sistemas Computacionales

Lenguajes y Autómatas I

Índice

1. Resumen del proyecto	3
2. Arquitectura y flujo de ejecución	3
3. Especificación del Lenguaje (tokens, reservadas y símbolos)	4
4. Gramática formal (EBNF)	6
5. Módulo lexer.py (analizador léxico)	7
5.1 Estructuras globales	7
5.2 Funciones y reglas del lexer	7
6. Módulo analizador_sintactico.py	11
6.1 Configuración y helpers	11
6.2 Recuperación antes del parse	11
6.3 Lexerproxy	12
6.4 modo pánico en p_error	13
6.5 API de alto nivel	14
6.6 Reglas de gramática (funciones p_*)	14
7. Módulo arbol_ast.py (árbol de sintaxis abstracta)	20
7.1 Métodos y funciones	21
8. Módulo analizador_lexico.py (GUI + Orquestación)	22
8.1 Variables y Widgets Principales	22
8.2 Funciones del módulo	22
9. Tabla de símbolos (Estructura y Reglas)	24
10. Clasificación de errores y mensajes (rúbrica)	25
12. Anexos: archivos generados por ply	26
VIDEO EXPLICACIÓN Y PRESENTACIÓN POWERPOINT	27

Documentación Del Compilador

1. Resumen del proyecto

Este proyecto consiste en el diseño e implementación de un Compilador para un **Lenguaje de Dominio Específico (DSL)** orientado a la programación de dispositivos de robótica educativa (tipo LEGO Mindstorms/Spike). El sistema está desarrollado en Python utilizando la herramienta **PLY (Python Lex-Yacc)** y abarca las fases de análisis léxico, sintáctico.

El núcleo del compilador se basa en una **Gramática Libre de Contexto** procesada mediante un algoritmo de análisis ascendente **LALR(1)**.

Adicionalmente, el sistema genera un **Árbol de Sintaxis Abstracta (AST)** visualizable mediante Graphviz, gestiona una tabla de símbolos para el chequeo de tipos y ofrece un entorno de desarrollo construido en **Tkinter**, que permite la edición, compilación y ejecución simulada de código en tiempo real.

2. Arquitectura y flujo de ejecución

Flujo general (de extremo a extremo):

- **Entrada (GUI):** El usuario ingresa el código fuente en el editor integrado (analizador_lexico.py), el cual gestiona la captura del texto y la interacción con los módulos de análisis.
- **Pre-Procesamiento Estructural:** Antes del análisis formal, se ejecuta una rutina de validación estructural (prechechar_delimitadores en analizador_sintactico.py). Esta fase realiza un escaneo preliminar para detectar bloques no cerrados (llaves faltantes) o instrucciones mal terminadas, evitando errores en cascada durante el parseo.
- **Análisis Léxico:** Si el pre-chequeo es exitoso, se invoca al lexer (lexer.py) para convertir el código fuente en una secuencia de tokens, identificando palabras reservadas, identificadores y operadores, y reportando errores léxicos.
- **Análisis Sintáctico y Construcción del AST:** El parser (analizador_sintactico.py) consume los tokens verificando el cumplimiento de la gramática LALR(1). En esta etapa:

1. Se valida la jerarquía y orden de las instrucciones.

- 2. Se recuperan errores no fatales.
- 3. Se construye dinámicamente el Árbol de Sintaxis Abstracta (AST).
- **Salida y Visualización:** Finalmente, el sistema genera el reporte de compilación en la consola de la GUI y, mediante `arbol_ast.py` y la herramienta Graphviz, renderiza gráficamente la estructura del código analizado.

3. Especificación del Lenguaje (tokens, reservadas y símbolos)

El léxico está implementado con `ply.lex`. El lenguaje reconoce palabras reservadas para tipos de dato, estructuras de control, funciones y dispositivos. Además define símbolos (operadores, delimitadores y puntuación) como tokens.

- Categorías principales:
 - **palabras reservadas:** INICIO, FIN, FUNC, RETORNAR, ENTERO, REAL, BOOLEANO, CADENA, SI, SINO, MIENTRAS, PARA, IMPRIMIR, RETRASO, ENCENDER, APAGAR, GIRAR SERVO, GIRAR MOTOR
 - **identificadores:** nombres de variables y funciones (token id).
 - **literales:** NUMERO (enteros), REAL (decimales), CADENA_TEXTO (entre comillas dobles), TRUE, FALSE.
 - **Operadores Aritméticos:** +, -, *, /, %, ++, --
 - **Operadores de Asignación:** =
 - **Operadores Lógicos y Relacionales:** &&, ||, !, ==, !=, <, >, <=, >=
 - **Acceso a Miembros:** -> (usado para invocar acciones en dispositivos, ej: motor -> encender()).
 - **Delimitadores:** {, }, (,), [,], ;, ,
 - **Dispositivos Soportados:**
Actuadores: SERVO, MOTOR, LED, BUZZER.

Sensores: SENSOR_ULTRASONICO, SENSOR_COLOR,
SENSOR_TOQUE, SENSOR_GIRO.

Hubs y Controladores: EV3, NXT, RCX, HUB_SPIKE_PRIME,
HUB_TECHNIC_LARGE

4. Gramática formal (EBNF)

La gramática base se describe en el archivo readme.txt y se implementa en analizador_sintactico.py.

GRAMATICA EN EBNF

```
programa          ::= "INICIO" bloque_codigo "FIN" ;
bloque_codigo     ::= "{" lista_declaraciones "}" ;
lista_declaraciones ::= lista_declaraciones declaracion
                    | lista_declaraciones control
                    | lista_declaraciones dispositivo
                    | lista_declaraciones funcion
                    | declaracion
                    | control
                    | dispositivo
                    | funcion ;
declaracion        ::= tipo ID ";"
                    | tipo ID ASIGNACION expresion ";"
                    | tipo ID CORCHETE_A NUMERO CORCHETE_B ";"
                    | ID CORCHETE_A NUMERO CORCHETE_B ASIGNACION expresion
                    ;
                    | IMPRIMIR "(" expresion ")" ";" ;
tipo               ::= "ENTERO"
                    | "REAL_TIPO"
                    | "BOOLEANO"
                    | "CADENA_TIPO" ;
tipo_dispositivo   ::= "SERVO"
                    | "LED"
                    | "MOTOR"
                    | "SENSOR"
                    | "BUZZER"
                    | "SENSOR_ULTRASONICO"
                    | "SENSOR_COLOR"
                    | "SENSOR_TOQUE"
                    | "SENSOR_GIRO" ;
control            ::= si
                    | mientras
                    | para
                    | romper ;
si                 ::= "SI" "(" expresion ")" bloque_codigo
                    | "SI" "(" expresion ")" bloque_codigo ID bloque_codigo
                    ;
mientras           ::= "MIENTRAS" "(" expresion ")" bloque_codigo ;
para               ::= "PARA" "(" asignacion_para ";" expresion ";"
                    | "PARA" "(" asignacion_para ";" expresion ";"
                    | "PARA" "(" asignacion_para ";" expresion ";"
                    ;
asignacion_para    ::= tipo ID ASIGNACION expresion
                    | ID ASIGNACION expresion ;
```

5. Módulo `lexer.py` (analizador léxico)

Responsabilidad: convertir el texto del programa en una secuencia de tokens (lexemas clasificados) y registrar errores léxicos. También expone utilidades para contexto/columna y una tabla de símbolos básica.

5.1 Estructuras globales

- Estructuras principales que se comparten con otros módulos:
 - `palabras_reservadas`: diccionario {lexema: token_type} para reconocer palabras clave.
 - `tokens`: tupla/lista de nombres de tokens consumidos por el parser.
 - `errores_lexicos`: lista de mensajes de error léxico generados durante el escaneo.
 - `errores_sintacticos`: lista compartida (se declara en `lexer.py` para que el parser la use como contenedor común).
 - `tabla_simbolos`: diccionario que almacena información de identificadores detectados (tipo, valor, línea, etc.).

5.2 Funciones y reglas del lexer

función: `t_comentario_linea`

propósito: ignora comentarios de una línea (`// ...`).

entradas: ninguna

salida/efecto: descarta el token (retorna `none`).

notas: implementada con reglas `ply.lex`; cuando es regla `t_x`, su `docstring/regex` define el patrón.

función: `t_newline`

propósito: actualiza el contador de líneas al encontrar saltos de línea.

entradas: token con `\n`

salida/efecto: incrementa `lexer.lineno`

notas: implementada con reglas `ply.lex`; cuando es regla `t_x`, su `docstring/regex` define el patrón.

función: `reset_errores_lexicos`

propósito: limpia la lista `errores_lexicos` para un nuevo análisis.

entradas: ninguna

salida/efecto: ninguna

notas: implementada con reglas ply.lex; cuando es regla t_x, su docstring/regex define el patrón.

función: encontrar_columna

propósito: calcula la columna de un token a partir de lexpos y el texto completo.

entradas: texto, token

salida/efecto: número de columna (iniciando el conteo en 1)

notas: implementada con reglas ply.lex; cuando es regla t_x, su docstring/regex define el patrón.

función: obtener_linea

propósito: devuelve el contenido de una línea específica del código fuente.

entradas: texto, numero de linea

salida/efecto: string (línea) o vacío.

notas: implementada con reglas ply.lex; cuando es regla t_x, su docstring/regex define el patrón.

función: puntero

propósito: genera '^' alineado para marcar la columna del error.

entradas: linea, columna

salida/efecto: string con espacios + '^'.

notas: implementada con reglas ply.lex; cuando es regla t_x, su docstring/regex define el patrón.

función: registrar_error_lexico

propósito: arma y registra un mensaje de error léxico con contexto (línea/columna).

entradas: mensaje base, codigo, token

salida/efecto: agrega a errores_lexicos.

notas: implementada con reglas ply.lex; cuando es regla t_x, su docstring/regex define el patrón.

función: formatear_contexto

propósito: devuelve un bloque de contexto: línea y puntero '^' en la columna.

entradas: código, línea, columna

salida/efecto: string multilínea.

notas: implementada con reglas ply.lex; cuando es regla t_x, su docstring/regex define el patrón.

función: t_CADENA_TEXTO

propósito: reconoce literales de cadena entre comillas dobles, con escapes simples.

entradas: (\"[^\"]*\").

salida/efecto: token CADENA_TEXTO.

notas: implementada con reglas ply.lex; cuando es regla t_x, su docstring/regex define el patrón.

función: t_REAL

propósito: reconoce literales reales con parte decimal (ej 2.5).

entradas: (\d+\.\d+)

salida/efecto: token REAL (float).

notas: implementada con reglas ply.lex; cuando es regla t_x, su docstring/regex define el patrón.

función: t_NUMERO

propósito: reconoce literales enteros.

entradas: (\d+).

salida/efecto: token NUMERO (int).

notas: implementada con reglas ply.lex; cuando es regla t_x, su docstring/regex define el patrón.

función: t_ID

propósito: reconoce identificadores y los mapea a palabras reservadas si aplica.

entradas: patrón `r'[a-zA-Z_][a-zA-Z0-9_]*'`

salida/efecto: token ID o token de reservada.

notas: implementada con reglas ply.lex; cuando es regla `t_x`, su docstring/regex define el patrón.

función: t_error

propósito: maneja caracteres ilegales no reconocidos por ninguna regla.

entradas: caracter actual

salida/efecto: registra error y avanza 1 char.

notas: implementada con reglas ply.lex; cuando es regla `t_x`, su docstring/regex define el patrón.

función: agregar_a_tabla_simbolos

propósito: registra un identificador en `tabla_simbolos` (si no existe o para actualizar).

entradas: id, info

salida/efecto: actualiza `tabla_simbolos`.

notas: implementada con reglas ply.lex; cuando es regla `t_x`, su docstring/regex define el patrón.

6. Módulo analizador_sintactico.py

Responsabilidad: validar la estructura del programa con una gramática libre de contexto usando ply.yacc, construir el AST y reportar errores sintácticos de forma robusta, evitando cascadas y continuando cuando sea posible (modo pánico).

6.1 Configuración y helpers

- Variables y helpers clave:
 - **max_errores_sintacticos:** limita la cantidad de errores para evitar saturación.
 - **_lineas_con_error:** conjunto para reportar a lo mucho 1 error por línea durante p_error.
 - **_codigo_actual:** copia del texto para construir contexto y columna en los errores.
 - **_lexer_actual:** referencia al lexerproxy activo para poder 'inyectar' tokens de recuperación.

6.2 Recuperación antes del parse

función: prechecar_delimitadores

propósito: hacer un pre-análisis sobre llaves/paréntesis/corchetes para detectar faltantes y desorden, intentando reportar todos los errores estructurales posibles.

entradas: codigo: str

salida/efecto: agrega mensajes a errores_sintacticos sin detener la ejecución.

estrategia: recorre tokens del lexer y mantiene una pila de aperturas esperadas para validar cierres; incluye reglas especiales como el patrón '}' fin' para cerrar root e identificar bloques internos sin cerrar.

función: validar_aperturas_bloque

propósito: validar casos donde ciertas palabras clave obligatoriamente deben abrir un bloque con '{' (ej: funcion(...){ ... }, si(...){...}).

entradas: codigo: str

salida/efecto: devuelve bool hubo_error y agrega mensajes claros a errores_sintacticos; no detiene el parse posterior.

6.3 Lexerproxy

clase: lexerproxy

propósito: envolver el lexer base para permitir 'pushback' de tokens y autocierre de delimitadores al final del archivo, ayudando a que el parser termine y pueda seguir reportando errores.

atributos: base: lexer real; _buf: tokens inyectados; _stack: cierres esperados.

método: lexerproxy.__init__

propósito: inicializa el wrapper con lexer base y estructuras internas.

entradas: depende del método; token() no recibe parámetros.

salida/efecto: retorna un LexToken real o artificial; puede modificar _buf/_stack.

método: lexerproxy.input

propósito: reinicia buffers/pila y pasa el texto al lexer base.

entradas: depende del método; token() no recibe parámetros.

salida/efecto: retorna un LexToken real o artificial; puede modificar _buf/_stack.

método: lexerproxy.lineno

propósito: sincroniza el número de línea con el lexer base.

entradas: depende del método; token() no recibe parámetros.

salida/efecto: retorna un LexToken real o artificial; puede modificar _buf/_stack.

método: lexerproxy.push_token

propósito: inyecta un token para que sea devuelto antes que los tokens reales

entradas: depende del método; token() no recibe parámetros.

salida/efecto: retorna un LexToken real o artificial; puede modificar _buf/_stack.

método: lexerproxy._fake

propósito: construye un token artificial (LexToken) para recuperación.

entradas: depende del método; token() no recibe parámetros.

salida/efecto: retorna un LexToken real o artificial; puede modificar _buf/_stack.

método: lexerproxy.token

propósito: retorna el siguiente token considerando: tokens inyectados, autocierre al EOF, inyección de '}' antes de FIN si faltan llaves, y mantenimiento de pila de aperturas/cierres.

entradas: depende del método; token() no recibe parámetros.

salida/efecto: retorna un LexToken real o artificial; puede modificar `_buf/_stack`.

método: lexerproxy.__getattr__

propósito: delegación: si se pide un atributo no definido, lo obtiene del lexer base.

entradas: depende del método; token() no recibe parámetros.

salida/efecto: retorna un LexToken real o artificial; puede modificar `_buf/_stack`.

6.4 modo pánico en p_error

función: _token_fake

propósito: crear un token artificial compatible con ply para inyectarlo al parser durante recuperación.

entradas: tipo, valor, lineno, lexpos

salida: LexToken con esos campos.

función: _panic_sync

propósito: descartar tokens hasta alcanzar un token de sincronización (punto y coma, llave de cierre o fin).

entradas: ninguna (consume tokens desde `parser.token()`)

salida/efecto: limpia el estado de error con `parser.errork()` y permite continuar.

función: p_error

propósito: manejador central de errores sintácticos. Reporta el error con línea/columna/contexto y aplica recuperación: sugerencias de reservadas, inserción de ';' faltantes o literales vacíos, y sincronización por pánico.

entradas: t: token problemático (o none al EOF)

salida/efecto: agrega mensajes a `errores_sintacticos` y retorna tokens artificiales cuando conviene para continuar.

6.5 API de alto nivel

función: `construir_parser`

propósito: construir el parser LR de ply.yacc y generar/actualizar `parsetab.py`.

entradas: ninguna

salida: objeto parser (yacc).

función: `parsear_codigo`

propósito: función pública que orquesta el análisis sintáctico: reinicia errores, ejecuta prechequeos y luego ejecuta `parser.parse` con `lexerproxy`. Finalmente unifica errores sin duplicados.

entradas: `codigo_fuente`: str

salida: retorna el AST (`NodoAST`) o `none` si falla; también llena `errores_sintacticos`.

6.6 Reglas de gramática (funciones `p_*`)

Cada función `p_*` implementa una producción de la gramática. En general: `p[0]` es el resultado (`NodoAST` o lista) y `p[1..n]` son símbolos de la producción.

regla: `p_programa`

qué valida: regla raíz: `programa -> inicio bloque_codigo fin`. crea nodo ast 'programa'.

entrada: símbolos en `p[1..n]` según la producción.

salida: `p[0]` (`NodoAST` o lista) o recuperación.

regla: `p_programa_inicio_mal`

qué valida: permite que el primer token sea ID (inicio mal escrito) para dar error y seguir.

entrada: símbolos en `p[1..n]` según la producción.

salida: `p[0]` (`NodoAST` o lista) o recuperación.

regla: `p_bloque_codigo`

qué valida: `bloque -> '{' lista_declaraciones '}'` (o error). maneja bloque no cerrado creando `bloque_incompleto`.

entrada: símbolos en `p[1..n]` según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_lista_declaraciones

qué valida: lista_declaraciones -> concatenación de declaraciones/controles/dispositivos/funciones.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_imprimir

qué valida: sentencia imprimir(expr); crea nodo 'imprimir'.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_retraso

qué valida: sentencia retraso(expr); crea nodo 'retraso'.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_declaracion_variable

qué valida: declara variables simples y arreglos, con o sin asignación.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_declaracion_asignacion_sin_expresion

qué valida: maneja 'tipo id =' como error recuperable.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_llamada_funcion

qué valida: llamada: id(args); crea nodo 'llamada_funcion'.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_tipo

qué valida: reduce tokens de tipo (entero, real, booleano, cadena).

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_control

qué valida: envuelve constructos de control (si, mientras, para, romper).

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_si

qué valida: si(cond) bloque [sino bloque].

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_mientras

qué valida: mientras(cond) bloque.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_para

qué valida: para(init; cond; update) bloque.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_asignacion_para

qué valida: parte init del for: tipo id = expr o id = expr.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_actualizacion_para

qué valida: parte update del for: id = expr, id++ o id--.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_funcion

qué valida: funcion id(params) bloque. crea nodo 'funcion'.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_parametros

qué valida: lista de parámetros (posible vacía).

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_parametro

qué valida: parametro individual: tipo id.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_dispositivo

qué valida: statement de dispositivos: definición o acción.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_tipo_dispositivo

qué valida: reduce el token del tipo de dispositivo (servo, led, motor, sensor, etc.).

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_definir_dispositivo

qué valida: tipo_dispositivo id = objeto_dispositivo;

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_objeto_dispositivo

qué valida: objeto dispositivo: '{' lista_atributos '}'.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_lista_atributos

qué valida: lista de pares atributo dentro del objeto de dispositivo.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_atributo

qué valida: atributo: id = expresion.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_accion_dispositivo

qué valida: id -> accion(expr_opt);

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_accion

qué valida: acciones permitidas: encender, apagar, valor, girarservo, girarmotor.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_expresion_opt

qué valida: expresión opcional (para acciones sin parámetros).

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_expresion_binaria

qué valida: expr op_binario expr. crea nodo 'binop'.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_expresion_unaria

qué valida: op_unario expr. crea nodo 'unop'.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_expresion_grupo

qué valida: (expr) para prioridad.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_expresion_valor_dispositivo

qué valida: id -> valor(expr_opt) usada como expresión.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_expresion_termino

qué valida: términos: id, id[idx], numero, real, cadena_texto, true/false.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_argumentos

qué valida: lista de argumentos (posible vacía).

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_op_binario

qué valida: reduce operadores binarios: +, -, *, /, %, ==, !=, <, >, <=, >=, and, or.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_op_unario

qué valida: reduce operadores unarios: not, - (negación).

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_romper

qué valida: romper; dentro de bucles.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_empty

qué valida: producción vacía.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

regla: p_error

qué valida: manejador de error descrito en 6.4.

entrada: símbolos en p[1..n] según la producción.

salida: p[0] (NodoAST o lista) o recuperación.

7. Módulo `arbol_ast.py` (árbol de sintaxis abstracta)

Responsabilidad: representar el AST con la clase `NodoAST` y ofrecer funciones para visualizar el árbol (por ejemplo, generando un `.dot` y renderizándolo).

clase: `NodoAST`

propósito: estructura de datos para representar nodos del ast (tipo de nodo + hijos + valor opcional).

campos típicos: tipo/etiqueta del nodo, valor, lista de hijos.

uso: el parser crea instancias para cada producción y luego se puede recorrer o graficar.

7.1 Métodos y funciones

método: `NodoAST.__init__`

propósito: crear un nodo con etiqueta y lista de hijos inicial (o vacía).

entradas: tipo: str, hijos: list[nodo] (opcional)

salida/efecto: instancia `NodoAST` lista para usarse en el parser.

método: `NodoAST.agregar_hijo`

propósito: agregar un hijo al nodo actual.

entradas: nodo hijo

salida/efecto: modifica la lista de hijos del nodo.

método: `NodoAST.__repr__`

propósito: representación en texto del nodo para depuración (print).

entradas: none

salida: string representando el nodo.

función: graficar_arbol

propósito: convertir el AST a un grafo en dot y renderizarlo (PNG) usando graphviz.

entradas: raíz del ast y nombre/archivo de salida (según implementación).

salida/efecto: genera archivos de salida para visualizar el árbol.

función: imprimir_arbol

propósito: imprimir una versión indentada del ast en consola/terminal (depuración).

entradas: nodo raíz

salida: texto en stdout.

8. Módulo analizador_lexico.py (GUI + Orquestación)

Responsabilidad: interfaz tkinter para editar código, lanzar análisis, mostrar tokens/tabla de símbolos/errores y ejecutar una simulación básica. Este archivo es el 'main' práctico del proyecto.

8.1 Variables y Widgets Principales

Se crean widgets para: editor de código (scrolledtext), consola de salida, panel de resultados y numeración de líneas. También se mantienen listas globales: errores_semanticos y avisos_lexicos.

8.2 Funciones del módulo

función: imprimir_en_lenguaje

propósito: escribe texto en la consola de ejecución de la gui.

entradas: depende: eventos tkinter o texto del editor.

salida/efecto: actualiza widgets y/o listas globales (errores, tabla de símbolos).

función: extraer_contenido

propósito: ver implementación; función de soporte de gui/análisis.

entradas: depende: eventos tkinter o texto del editor.

salida/efecto: actualiza widgets y/o listas globales (errores, tabla de símbolos).

función: analizar_codigo

propósito: función principal: corre lexer+parser, tokens, tabla símbolos y muestra resultados.

entradas: depende: eventos tkinter o texto del editor.

salida/efecto: actualiza widgets y/o listas globales (errores, tabla de símbolos).

función: procesar_funciones

propósito: ver implementación; función de soporte de gui/análisis.

entradas: depende: eventos tkinter o texto del editor.

salida/efecto: actualiza widgets y/o listas globales (errores, tabla de símbolos).

función: clonar_tokens

propósito: ver implementación; función de soporte de gui/análisis.

entradas: depende: eventos tkinter o texto del editor.

salida/efecto: actualiza widgets y/o listas globales (errores, tabla de símbolos).

función: procesar_tokens

propósito: recorre tokens para clasificar/contar y alimentar tabla de símbolos.

entradas: depende: eventos tkinter o texto del editor.

salida/efecto: actualiza widgets y/o listas globales (errores, tabla de símbolos).

función: manejar_declaracion

propósito: registra declaraciones de variables/arrays y valida redeclaración.

entradas: depende: eventos tkinter o texto del editor.

salida/efecto: actualiza widgets y/o listas globales (errores, tabla de símbolos).

función: mostrar_resultados

propósito: pinta en pantalla: tokens, tabla de símbolos, errores léxicos/sintácticos/semánticos.

entradas: depende: eventos tkinter o texto del editor.

salida/efecto: actualiza widgets y/o listas globales (errores, tabla de símbolos).

función: ejecutar_codigo

propósito: interpreta de forma muy básica: imprimir(...) y algunas acciones simuladas.

entradas: depende: eventos tkinter o texto del editor.

salida/efecto: actualiza widgets y/o listas globales (errores, tabla de símbolos).

función: verificar_variables_usadas

propósito: revisa uso de variables no declaradas (error semántico).

entradas: depende: eventos tkinter o texto del editor.

salida/efecto: actualiza widgets y/o listas globales (errores, tabla de símbolos).

función: actualizar_numeros_linea

propósito: actualiza el panel de numeración al cambiar el texto.

entradas: depende: eventos tkinter o texto del editor.

salida/efecto: actualiza widgets y/o listas globales (errores, tabla de símbolos).

función: on_scrollbar

propósito: sincroniza scroll de editor y numeración.

entradas: depende: eventos tkinter o texto del editor.

salida/efecto: actualiza widgets y/o listas globales (errores, tabla de símbolos).

función: on_textscroll

propósito: evento de scroll del editor.

entradas: depende: eventos tkinter o texto del editor.

salida/efecto: actualiza widgets y/o listas globales (errores, tabla de símbolos).

9. Tabla de símbolos (Estructura y Reglas)

La tabla de símbolos se usa para registrar identificadores (variables, dispositivos, funciones) con metadatos que permiten validaciones semánticas simples. Se alimenta en `lexer.py` y también en `analizador_lexico.py` cuando se recorren tokens para capturar tipo, valor y redeclaraciones.

- Campos típicos recomendados en cada entrada:
 - nombre (lexema)
 - clase: variable | arreglo | funcion | dispositivo
 - tipo: entero | real | booleano | cadena | tipo_dispositivo
 - valor actual (si aplica)
 - linea de declaración
 - notas (por ejemplo: tamaño de arreglo, atributos del dispositivo, etc.)

10. Clasificación de errores y mensajes (rúbrica)

El sistema separa errores por etapas:

- Léxicos: caracteres ilegales, cadena mal formada, número mal formado, identificador con símbolo inválido.
- Sintácticos: estructura inválida con respecto a la gramática (faltan ';', faltan llaves, tokens fuera de lugar).

Codigo Error	mensaje que sale	tipo de error	Que significa
501	posible falta de ';' al final de la línea anterior	sintactico	la línea anterior no terminó con ';' y por eso la siguiente instrucción se interpreta como error
502	falta una expresion despues de '=' (no puedes dejar la asignacion vacia)	sintactico	hiciste una asignación sin valor (ej: entero x = ;)
503	x error de sintaxis en '___' (token ___)	sintactico	el parser encontró un token inesperado según la gramática (puede venir por ; faltante, paréntesis/llaves faltantes, etc.)
504	x error de sintaxis: fin de archivo inesperado (posible falta de '}' o '}')	sintactico	el archivo terminó y quedó algo abierto (normalmente falta cerrar '}' o '}')
505	x error de sintaxis: falta '}' antes de declarar nueva funcion	sintactico	estás declarando una función cuando aún hay un bloque interno abierto; por eso "falta } antes de declarar nueva función"
506	x error de sintaxis: falta '}' para cerrar el bloque '{' iniciado en línea ___	sintactico	antes de terminar el programa se detectó que quedó un bloque interno sin cerrar con '}'
507	x error de sintaxis: antes de 'fin' debes cerrar el bloque principal con '}'	sintactico	llegaste a 'fin' con el bloque principal aún abierto; falta cerrar con '}'
508	x error de sintaxis: falta cerrar '}' o '}' en esta línea (se encontro '{' con '{' o '{' al sintactico	sintactico	apareció '}' pero todavía había un '{' o '{' abierto en esa misma línea; primero debías cerrar '}' o '}'
509	x error de sintaxis: '}' inesperado sin apertura	sintactico	apareció un cierre (}), } o }) sin que exista su apertura correspondiente (faltó abrir o sobró un cierre)
510	x error de sintaxis: se esperaba '___' pero se encontro '___'	sintactico	cerraste con un símbolo distinto al esperado (ej: se esperaba '}' pero se encontró '}', etc.)
511	x error de sintaxis: antes de 'fin' debes cerrar el bloque principal con '}' (se encontro sintactico	sintactico	caso especial: antes de 'fin' apareció '}' pero el programa debe cerrar con '}' fin'
512	x error de sintaxis: Bloque no cerrado al final del archivo (falta '___')	sintactico	al final todavía quedaron aperturas pendientes; falta el cierre indicado
513	x error de sintaxis: despues de 'inicio' se esperaba abrir llave '{'	sintactico	después de 'inicio' no abriste '{' como exige la estructura del programa
514	x error de sintaxis: falta abrir llave '{' para el bloque de 'func/si/mientras/para	sintactico	en func/si/mientras/para terminaste el '}' pero no abriste '{' para el bloque
515	x error de sintaxis: se esperaba 'inicio' pero se encontro '___'	sintactico	la primera palabra del programa no fue 'inicio' (mal escrita o no está donde debe)
516	x error de sintaxis: se esperaba 'sino' pero se encontro '___'	sintactico	en un si {...} {} la parte "sino" está mal escrita o no coincide con lo que espera la gramática
517	x error de sintaxis: Bloque no cerrado. Falta '}' para cerrar el bloque iniciado en sintactico	sintactico	se detectó un bloque empezado con '{' que no cerró con '}' durante el parse (recuperación)
601	~ aviso: identificador '___' (línea __, columna __). ¿quisiste decir '___'?	lexico (aviso)	aviso: escribiste un identificador parecido a una reservada y te sugiere la correcta (no bloquea)
701	x error semantico: variable '___' usada sin declaracion (línea __, columna __)	semantico	usaste una variable que no está declarada antes (no existe en la tabla de símbolos)
702	x error semantico: tipo de dato incorrecto en '___'	semantico	declaraste una variable con un tipo y le asignaste un valor de otro tipo incompatible
801	no se pudo construir el arbol sintactico debido a errores	sintactico (res)	hubo errores sintácticos y por eso no se generó el árbol

11. Guía de ejecución

- Requisitos:
 - python 3.x
 - ply (lex/yacc)
 - tkinter (incluido en muchas distribuciones de python)
 - graphviz instalado si quieres generar imágenes del ast

- Ejecución típica:
 - ejecuta analizador_lexico.py (es la GUI).
 - pega un programa en el editor.
 - usa los botones de analizar / ejecutar / graficar (según la GUI).

12. Anexos: archivos generados por ply

El sistema genera automáticamente archivos auxiliares esenciales para el funcionamiento y depuración del parser LALR(1):

- **parsetab.py (Caché de Tablas):** Contiene la representación compilada de la tabla de autómatas finitos y transiciones LR. PLY lo genera para optimizar el tiempo de arranque, evitando recalcular la gramática en cada ejecución a menos que el código fuente cambie.
- **parser.out (Bitácora de Depuración):** Es un archivo de texto plano fundamental para el desarrollador. Detalla:
 - La gramática completa reconocida.
 - Los estados del autómata generados.
 - Los conflictos Shift/Reduce o Reduce/Reduce (si existieran), permitiendo diagnosticar ambigüedades en el diseño del lenguaje.

VIDEO EXPLICACIÓN Y PRESENTACIÓN POWERPOINT

VIDEO

 compilador.mp4

PRESENTACIÓN

 MINI-ROBOTCODE.pdf

ARCHIVOS (CODIGO)

https://drive.google.com/file/d/1_Y7_p6AHJwn0YFrWLt3Fw1fvHm3aeaMO/view?usp=sharing